DCRM — Dependency Continuity & Risk Module

DCRM — Dependency Continuity & Risk Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Dependency Standard (SDS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Dependency Continuity & Risk Governance

Module ID: SLGA-SDS-DCRM-005

Origin ID: OOF.CM.SLGA.03.05.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Dependency Continuity & Risk Module (DCRM) governs the recognition,

assessment, preservation, and reassessment of operational continuity and material risk arising from software dependency relationships throughout the software lifecycle.

DCRM establishes the methodological conditions under which a software

dependency may be evaluated for its operational criticality, availability, maintenance condition, support dependency, concentration, substitutability, failure exposure, degradation consequences, recovery dependencies, and continuity limitations.

It governs the relationship between a dependent software object, its

material dependency resources, the operational functions supported by those resources, the conditions required for continued reliance, the consequences of dependency disruption, and the evidence supporting continuity and risk determinations.

DCRM distinguishes current dependency functionality from sustainable

operational reliance, dependency availability from continuity assurance, the existence of alternatives from effective substitutability, and recognized risk from demonstrated risk control.

It applies to internal and external software components, libraries,

packages, frameworks, plugins, runtime platforms, software services, build resources, generated components, embedded dependencies, and material direct or transitive dependency relationships.

DCRM applies regardless of whether a dependency was introduced by human

developers, organizations, automated development tools, AI coding assistants, autonomous software development agents, suppliers, or software infrastructure mechanisms.

Its principal output is:

Governed Dependency Continuity & Risk State

Its foundational distinction is:

Current Dependency Functionality ≠ Governed Dependency Continuity.

Operational Role

The operational role of DCRM is:

To ensure that material software dependencies remain subject to evidence-supported assessment of their criticality, availability, maintenance, concentration, substitutability, disruption exposure, and operational continuity, so that continued software reliance does not depend on unrecognized or unjustified assumptions about the resources upon which the software operates.

DCRM establishes the continuity and risk-governance foundation of SDS.

It evaluates whether the dependency relationships identified, traced,

assessed for suitability, and introduced under applicable authority can continue supporting the intended software context under relevant operational conditions.

DCRM does not independently guarantee availability, eliminate dependency failure, certify software security, establish business continuity for the entire organization, or determine legal liability.

It provides a governed dependency-specific continuity and risk state for

competent software lifecycle decisions.

Operational Question

Under which evidenced operational, maintenance, availability,

substitutability, and risk conditions can a software system continue relying on a material dependency, and how can that reliance remain governable when the dependency degrades, changes, becomes unsupported, or fails?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

DCRM operates within:

Software Dependency Standard (SDS)

The SDS internal governance flow is:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

DCRM governs:

Dependency Continuity & Risk

It is the fifth Core Module of SDS.

Its principal upstream inputs are:

Governed Software Component Identification State

from:

CIM — Component Identification Module

Governed Dependency Origin & Traceability State

from:

DOTM — Dependency Origin & Traceability Module

Governed Dependency Suitability State

from:

DSM — Dependency Suitability Module

Governed Component Introduction & Change State

from:

CICM — Component Introduction & Change Module

Its principal output is:

Governed Dependency Continuity & Risk State

This output supports the parent SDS governance determination:

Governed Software Dependency State

and competent downstream SLGA™ governance.

DCRM may be invoked repeatedly whenever dependency conditions, operational exposure, support arrangements, or software lifecycle requirements materially change.

• Governed Space

Software Dependency Continuity & Risk Governance

This space governs the sustainability and operational risk of material

software dependency relationships.

Its principal dimensions include:

• Dependency continuity governance objects

• Dependent software functions

• Dependency operational roles

• Dependency criticality

• Dependency failure consequences

• Dependency availability

• Availability assumptions

• Maintenance condition

• Support condition

• Dependency lifecycle condition

• Dependency concentration

• Shared dependency exposure

• Direct and transitive failure propagation

• External service reliance

• Supplier and provider dependency

• Dependency substitutability

• Replacement feasibility

• Migration constraints

• Recovery dependency

• Dependency degradation

• Dependency failure

• Continuity requirements

• Continuity evidence

• Risk identification

• Risk assessment

• Risk treatment references

• Residual risk

• Continuity uncertainty

• Continuity conflicts

• Continuity gaps

• Critical continuity gaps

• Dependency continuity drift

• Continuity reassessment

• Dependency continuity governance states

The governed space is independent of any particular software technology, package registry, supplier, operating platform, or development methodology.

Dependency Continuity & Risk Governance Object

Every DCRM assessment shall establish a Dependency Continuity & Risk

Governance Object.

The object represents the continuity and risk conditions of a specific

software dependency relationship.

It should identify:

Governance Object Identity: The identifiable continuity and risk

assessment.

Dependent Software Object: The software system relying on the dependency.

Dependency Component: The relevant component or software resource.

Dependency Relationship: The material reliance relationship.

Operational Function: The function supported by the dependency.

Dependency Criticality: The significance of the dependency to the

dependent software.

Continuity Requirements: The conditions required for continued reliance.

Availability Condition: The relevant availability assumptions and

evidence.

Maintenance Condition: The dependency's maintenance and support status.

Concentration Exposure: Material reliance on shared or concentrated

resources.

Substitutability Condition: The feasibility of replacing the dependency.

Disruption Exposure: Material consequences of degradation or failure.

Recovery Dependencies: Resources and conditions required for restoration.

Continuity Evidence: Evidence supporting the assessment.

Continuity Uncertainty: Material unresolved continuity conditions.

Risk Condition: The recognized dependency-specific risk.

Governance State: The current continuity and risk condition.

The governance object shall remain distinguishable from the dependency

component and the complete software system.

Dependency Continuity Governance Object ≠ Software Dependency Artifact.

Dependent Software Function Identification

DCRM shall identify the software functions materially supported by the

dependency.

Relevant functions may include:

• Core application execution

• Data processing

• Authentication

• Communication

• Storage

• Transaction processing

• User interaction

• Monitoring

• Software build and recovery

• System integration

• Operational coordination

A dependency may support multiple functions.

A function may depend on multiple resources.

Dependency Function ≠ Dependency Criticality.

Dependency Operational Role

DCRM shall establish the operational role of the dependency within the dependent software context.

Roles may include:

• Essential execution resource

• Supporting runtime resource

• External service provider

• Build and recovery resource

• Optional functionality provider

• Integration component

• Shared platform dependency

• Conditional operating resource

The role shall be evaluated in relation to actual software reliance.

Technical Component Classification ≠ Operational Dependency Role.

Dependency Criticality

DCRM shall assess the significance of a dependency to the continued

operation of the dependent software.

Criticality may depend on:

• Supported software functions

• Consequences of dependency failure

• Availability of alternatives

• Failure propagation

• Recovery requirements

• Operational time sensitivity

• Dependency concentration

• Relevant service obligations

A small dependency may support a critical function.

A large dependency may support only optional functionality.

Dependency Size ≠ Dependency Criticality.

Criticality Context

DCRM shall establish the context within which dependency criticality is assessed.

Relevant contexts may include:

• Development

• Production

• Maintenance

• Recovery

• Safety-relevant operation

• Business-critical operation

• Public-service operation

• Autonomous system execution

Criticality shall not be assumed to remain identical across all contexts.

Dependency Criticality in One Context ≠ Universal Dependency Criticality.

Dependency Failure Consequences

DCRM shall identify material consequences of dependency failure or

degradation.

Consequences may include:

• Loss of functionality

• Reduced performance

• Data-processing interruption

• Communication failure

• Authentication failure

• Service unavailability

• Build failure

• Recovery impairment

• Propagation to dependent components

Consequences shall be assessed relative to the governed software context.

Dependency Failure ≠ Uniform Operational Consequence.

Dependency Availability

DCRM shall assess the availability conditions required for continued

dependency reliance.

Availability may concern:

• Package accessibility

• Runtime component availability

• External service availability

• Platform availability

• Repository availability

• Build resource availability

• Required infrastructure availability

Current availability shall not independently establish future availability.

Current Dependency Availability ≠ Continuity Assurance.

Availability Assumptions

DCRM shall identify material assumptions concerning dependency availability.

Assumptions may include:

• Continued service operation

• Continued package distribution

• Continued platform access

• Stable supplier arrangements

• Continued infrastructure support

• Continued access to required build resources

Unsupported assumptions shall remain explicit.

Assumed Availability ≠ Evidenced Availability.

Maintenance Condition

DCRM shall identify the maintenance condition of material software

dependencies.

Relevant conditions may include:

• Actively maintained

• Limited maintenance

• Maintenance uncertain

• Unsupported

• Abandoned

• End-of-life

• Maintenance transferred

• Internally maintained

Maintenance status shall be evaluated using relevant evidence.

A dependency may continue functioning after maintenance has ended.

Functional Dependency ≠ Maintained Dependency.

Support Condition

DCRM shall assess relevant support relationships affecting continued

dependency reliance.

Support may involve:

• Original developers

• Open-source maintainers

• Commercial suppliers

• Internal maintenance teams

• Platform providers

• Managed service operators

• Third-party support organizations

The existence of a support provider shall not independently establish

adequate support capability.

Support Provider Existence ≠ Effective Support Assurance.

Dependency Lifecycle Condition

DCRM shall identify material lifecycle conditions affecting a dependency.

Conditions may include:

• Active development

• Stable maintenance

• Deprecated

• Maintenance-only

• Unsupported

• End-of-life

• Replacement planned

• Retirement pending

Lifecycle condition shall remain distinguishable from immediate technical

functionality.

Dependency Lifecycle Status ≠ Current Functional Status.

Dependency Maintenance Evidence

DCRM shall preserve evidence supporting maintenance and support claims.

Evidence may include:

• Maintenance announcements

• Release histories

• Support agreements

• Supplier notices

• Project activity records

• End-of-life notices

• Internal maintenance arrangements

• Relevant service documentation

A recent software release shall not independently establish sustainable

future maintenance.

Recent Release Activity ≠ Long-Term Maintenance Assurance.

Dependency Concentration

DCRM shall identify material concentration of software reliance.

Concentration may arise where:

• Multiple systems depend on one component

• Multiple applications rely on one provider

• Several services share one runtime platform

• Many dependency paths converge on one library

• Multiple recovery processes require the same external resource

Concentration may increase the consequences of a single dependency disruption.

Multiple Dependency Relationships ≠ Independent Dependency Resilience.

Shared Dependency Exposure

DCRM shall assess exposure arising from shared dependencies across software

systems.

A shared dependency may affect:

• Multiple applications

• Multiple business functions

• Multiple software environments

• Multiple autonomous agents

• Multiple infrastructure services

The same component may create correlated failure exposure.

System Separation ≠ Dependency Independence.

Transitive Dependency Risk

DCRM shall identify material risk arising from transitive dependency

relationships.

A dependency may be operationally critical even when the dependent application does not reference it directly.

Relevant assessment dimensions include:

• Dependency chain position

• Supported functions

• Failure propagation

• Replacement feasibility

• Maintenance condition

• Evidence limitations

Indirect Dependency ≠ Indirect Operational Consequence.

Dependency Failure Propagation

DCRM shall assess how dependency disruption may propagate through the dependent software.

A relevant chain may be:

Dependency Failure → Component Degradation → Software Function Disruption

→ Operational Consequence

Failure propagation shall be evaluated relative to actual dependency

relationships.

The existence of a dependency chain does not independently establish that failure will propagate through every component.

Dependency Connection ≠ Proven Failure Propagation.

External Service Dependency Risk

DCRM shall assess continuity exposure arising from external software

services.

Relevant conditions may include:

• Provider availability

• Service interface changes

• Access restrictions

• Service termination

• Contractual service conditions

• Operational concentration

• Network dependency

• Provider-specific integration

A service may be operationally critical without being installed as a software package.

No Installed Component ≠ No External Dependency Risk.

Supplier Dependency Risk

DCRM shall identify material continuity exposure arising from software

suppliers and providers.

Relevant conditions may include:

• Supplier support

• Supplier concentration

• Distribution control

• Maintenance dependence

• Service availability

• Provider replacement feasibility

• Material supplier changes

Supplier dependency risk shall remain distinguishable from proof of

supplier misconduct or failure.

Supplier Reliance ≠ Proven Supplier Failure.

Dependency Substitutability

DCRM shall assess whether a dependency can be effectively replaced within the intended software context.

Substitutability may depend on:

• Functional equivalence

• Interface compatibility

• Data compatibility

• Migration complexity

• Required development effort

• Available alternatives

• Operational transition conditions

• Replacement authorization

• Testing requirements

• Recovery constraints

The existence of another component shall not independently establish

effective substitutability.

Alternative Availability ≠ Effective Dependency Substitutability.

Replacement Feasibility

DCRM shall assess the practical feasibility of replacing a material

dependency.

Relevant conditions may include:

• Technical migration requirements

• Required software modifications

• Configuration changes

• Data migration

• Integration changes

• Testing requirements

• Operational interruption

• Required expertise

• Available resources

Replacement feasibility shall remain distinguishable from permission to

execute replacement.

Technically Feasible Replacement ≠ Authorized Dependency Replacement.

Dependency Lock-In

DCRM shall identify material conditions that make continued software operation dependent on a difficult-to-replace component, service, platform, or supplier.

Lock-in may arise through:

• Proprietary interfaces

• Specialized data formats

• Unique runtime requirements

• Non-portable integrations

• Concentrated supplier control

• High migration complexity

• Limited alternatives

• Embedded platform dependencies

Lock-in shall be assessed as a dependency relationship condition rather

than inferred solely from supplier identity.

Supplier Dependence ≠ Automatically Unacceptable Lock-In.

Dependency Recovery Requirements

DCRM shall identify material conditions required to restore software

functionality following dependency disruption.

Requirements may include:

• Replacement component availability

• Previous artifact availability

• Configuration restoration

• Compatible runtime environments

• Required build tools

• Recovery documentation

• Operational access

• Competent intervention

A recovery plan shall not be treated as demonstrated recovery capability

without sufficient supporting evidence.

Documented Recovery Plan ≠ Verified Recovery Capability.

Recovery Dependency Identification

DCRM shall identify dependencies required specifically for software

recovery.

These may include:

• Build repositories

• Package registries

• Backup services

• Deployment tools

• Authentication services

• Recovery platforms

• Required software licenses

• Specialized maintenance tools

A software system may be operationally independent of a resource during normal execution but dependent upon it during recovery.

Runtime Dependency Set ≠ Recovery Dependency Set.

Dependency Degradation

DCRM shall identify material dependency degradation conditions.

Degradation may include:

• Reduced performance

• Increased error rates

• Partial service unavailability

• Interface instability

• Maintenance deterioration

• Reduced support

• Resource exhaustion

• Intermittent component failure

A degraded dependency may continue functioning while its continuity condition deteriorates.

Continued Functionality ≠ Absence of Dependency Degradation.

Dependency Failure

DCRM shall recognize material dependency failure conditions.

Failure may concern:

• Component execution

• Service availability

• Interface operation

• Package retrieval

• Platform compatibility

• Build capability

• Recovery capability

Failure recognition shall distinguish observed failure from inferred

failure causes.

Observed Dependency Failure ≠ Verified Failure Cause.

Dependency Risk Identification

DCRM shall identify material risks associated with continued dependency

reliance.

Relevant risk categories may include:

• Availability risk

• Maintenance risk

• Support risk

• Compatibility risk

• Concentration risk

• Supplier dependency risk

• Transitive dependency risk

• Replacement risk

• Recovery risk

• Lifecycle discontinuity risk

Security-specific risks shall be referred to competent software security

governance.

Recognized Dependency Risk ≠ Demonstrated Risk Control.

Dependency Risk Assessment

DCRM shall assess material dependency risks relative to the dependent

software context.

Assessment may consider:

• Plausible disruption conditions

• Relevant exposure

• Potential consequences

• Dependency criticality

• Existing controls

• Available evidence

• Recovery conditions

• Material uncertainty

Where probability or impact estimates are used, their assumptions and evidentiary basis shall remain explicit.

Risk Score ≠ Complete Dependency Risk Understanding.

Dependency Risk Prioritization

DCRM shall support prioritization of material dependency risks.

Prioritization may consider:

• Criticality

• Disruption consequences

• Evidence-supported exposure

• Available controls

• Replacement difficulty

• Recovery limitations

• Concentration

• Time sensitivity

Priority shall not be determined solely by component size, popularity, or

numerical scoring.

High Dependency Popularity ≠ Low Dependency Risk.

Dependency Risk Treatment Reference

DCRM shall preserve relevant risk-treatment conditions.

Possible treatments may include:

• Continued monitoring

• Additional maintenance arrangements

• Alternative dependency preparation

• Dependency replacement planning

• Recovery preparation

• Reduced dependency scope

• Technical isolation

• Additional evidence collection

• Competent escalation

DCRM governs the dependency-specific risk-treatment reference.

Execution authority and broader lifecycle change governance remain within their competent domains.

Risk Treatment Proposal ≠ Authorized Risk Treatment Execution.

Residual Dependency Risk

DCRM shall identify material risks remaining after relevant controls or

treatment measures.

Residual risk shall reflect:

• Remaining exposure

• Control limitations

• Unresolved dependencies

• Recovery constraints

• Material uncertainty

• Relevant operational consequences

The presence of a risk treatment shall not independently establish that the risk has been adequately reduced.

Implemented Control ≠ Acceptable Residual Risk.

Dependency Risk Acceptance Reference

DCRM shall preserve references to competent risk-acceptance decisions where applicable.

Risk acceptance shall remain distinguishable from risk identification and

assessment.

A risk assessment does not independently authorize continued reliance under every condition.

Risk Assessment ≠ Risk Acceptance Authority.

Continuity Requirements

DCRM shall establish material continuity requirements for the dependency

relationship.

Requirements may concern:

• Required availability

• Permitted degradation

• Recovery expectations

• Maintenance conditions

• Support conditions

• Replacement readiness

• Dependency monitoring

• Operational constraints

Requirements shall reflect the governed software context.

General Availability Expectation ≠ Defined Continuity Requirement.

Dependency Continuity Evidence

DCRM shall preserve evidence supporting continuity and risk claims.

Evidence may include:

• Availability records

• Maintenance records

• Support agreements

• Recovery tests

• Dependency inventories

• Operational observations

• Supplier notices

• Replacement assessments

• Failure records

• Incident histories

• Relevant technical evaluations

Evidence shall remain associated with the specific continuity claim it

supports.

Continuity Evidence Presence ≠ Continuity Claim Validity.

Continuity Evidence Competence

DCRM shall assess whether available evidence is competent to support the relevant continuity determination.

For example:

An availability record may support historical availability.

A maintenance agreement may support a defined support obligation.

A recovery test may support demonstrated recovery under tested conditions.

A replacement assessment may support technical substitution feasibility.

These records establish different facts.

Historical Availability Evidence ≠ Future Availability Guarantee.

Continuity Evidence Integrity

DCRM shall preserve the integrity of material continuity and risk evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Software version correspondence

• Operating environment correspondence

• Supplier correspondence

• Material contradictions

• Evidence completeness

Evidence integrity does not independently establish sustainable continuity.

Intact Continuity Evidence ≠ Complete Continuity Assurance.

Dependency Continuity Validation

DCRM shall determine whether material continuity conditions are sufficiently supported within the assessed scope.

Validation should consider:

• Dependency criticality

• Availability requirements

• Maintenance condition

• Support condition

• Concentration

• Failure propagation

• Substitutability

• Recovery dependencies

• Risk exposure

• Evidence competence

• Material uncertainty

Validation shall remain limited to the conditions actually assessed.

Validated Continuity Conditions ≠ Guaranteed Dependency Availability.

Dependency Continuity Uncertainty

Continuity uncertainty may arise where:

• Future support is unknown

• Supplier availability is uncertain

• Maintenance status is unclear

• Replacement feasibility is unresolved

• Recovery capability has not been demonstrated

• Critical dependency relationships remain unidentified

• Concentration exposure is incomplete

• Relevant risk evidence is insufficient

DCRM shall preserve material uncertainty.

Uncertain Continuity ≠ Proven Imminent Failure.

Dependency Continuity Conflict

A continuity conflict exists where material evidence supports incompatible continuity conclusions.

Examples may include:

• Conflicting supplier support claims

• Contradictory maintenance records

• Inconsistent availability evidence

• Conflicting recovery test results

• Incompatible replacement feasibility assessments

DCRM shall preserve the conflict and its supporting evidence.

Conflicting Continuity Evidence ≠ Permission to Select the Preferred

Conclusion.

Dependency Continuity Gap

A Dependency Continuity Gap exists where a material condition required for competent continuity assessment remains insufficiently established.

Examples may include:

• Missing maintenance evidence

• Unknown supplier support

• Unverified replacement feasibility

• Missing recovery dependency identification

• Unassessed concentration

• Unresolved failure propagation

• Incomplete availability evidence

DCRM shall identify the gap and its material significance.

Continuity Gap ≠ Proven Dependency Failure.

Critical Dependency Continuity Gap

A Critical Dependency Continuity Gap exists where insufficient continuity evidence prevents competent assessment of a materially consequential software dependency.

Examples may include:

- A critical external service without a sufficiently established

continuity arrangement. - An essential unsupported library without an assessed replacement path. - A critical shared platform with unresolved concentration exposure. - A recovery-critical package repository whose availability cannot be established.

DCRM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, security compromise, or legal liability.

Critical Continuity Gap ≠ Automatic Operational Failure.

Dependency Continuity Drift

Dependency Continuity Drift occurs when actual dependency conditions materially diverge from the conditions under which continuity was previously assessed.

Possible causes include:

• Maintenance termination

• Supplier changes

• Service degradation

• Dependency updates

• Platform migrations

• New transitive dependencies

• Increased operational reliance

• Reduced substitutability

• Changed recovery requirements

• Concentration growth

DCRM shall identify material drift and support reassessment.

Previously Acceptable Dependency Risk ≠ Permanently Acceptable Dependency

Risk.

Dependency Continuity Change

DCRM shall preserve material changes in dependency continuity conditions.

Changes may include:

• Updated maintenance status

• New support arrangements

• Changed criticality

• Revised availability evidence

• New replacement options

• Changed recovery capability

• New concentration exposure

• Updated risk assessments

• Resolved continuity gaps

Historical continuity states shall remain reconstructable where material.

Continuity Reassessment ≠ Historical Risk Erasure.

Dependency Continuity Monitoring

DCRM shall support monitoring of material dependency conditions where monitoring is justified by criticality and risk.

Monitoring may concern:

• Availability

• Maintenance announcements

• Support status

• End-of-life notices

• Dependency changes

• Service degradation

• Supplier changes

• Replacement readiness

• Recovery dependencies

• Critical risk indicators

Monitoring shall remain proportional to the dependency's operational

significance.

Monitoring Activity ≠ Effective Continuity Control.

Dependency Continuity Reassessment

DCRM shall support reassessment when material continuity conditions change.

Triggers may include:

• Dependency failure

• Service degradation

• Maintenance termination

• End-of-life announcements

• Supplier changes

• Critical dependency updates

• New transitive dependencies

• Platform changes

• Recovery failures

• Changed operational requirements

• Increased concentration

• New continuity evidence

• Material continuity drift

Reassessment shall preserve material historical continuity records.

Dependency Continuity & Risk Governance States

DCRM shall support identifiable governance states.

Possible states include:

Under Assessment: Continuity and risk conditions are being evaluated.

Continuity-Supported: Material continuity conditions are sufficiently supported within the assessed scope.

Conditionally Supported: Continued reliance depends on explicit

conditions.

Continuity-at-Risk: Material conditions threaten continued dependency

reliance.

Maintenance-Unresolved: Maintenance status cannot be sufficiently

established.

Support-Unresolved: Material support conditions remain uncertain.

Availability-Unresolved: Relevant availability conditions remain

insufficiently established.

Substitutability-Unresolved: Effective replacement feasibility remains

uncertain.

Recovery-Unresolved: Material recovery conditions remain insufficiently

established.

Evidence-Limited: Available evidence is insufficient for the required determination.

Conflicted: Material continuity evidence is contradictory.

Continuity-Unsupported: Material continuity requirements are not satisfied within the assessed scope.

Disrupted: Material dependency functionality or availability has been

interrupted.

Recovery-Pending: Relevant recovery conditions remain unresolved.

Superseded: A later governance state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not independently establish software release readiness,

legal liability, or organizational business continuity.

Continuity-Supported ≠ Dependency Failure Impossible.

Governed Dependency Continuity & Risk State

The principal output of DCRM is:

Governed Dependency Continuity & Risk State

The state shall preserve sufficient information to identify:

Continuity Governance Object: The dependency continuity and risk

assessment.

Dependent Software Object: The software system relying on the dependency.

Component Identity: The relevant dependency component.

Dependency Relationship: The material software reliance relationship.

Operational Function: The function supported by the dependency.

Operational Role: The dependency's role in continued software operation.

Dependency Criticality: The significance of dependency disruption.

Criticality Context: The environment and operational context of the

assessment.

Failure Consequences: Material effects of dependency disruption.

Availability Condition: The recognized availability state.

Availability Assumptions: Material assumptions concerning continued

access.

Maintenance Condition: The relevant maintenance status.

Support Condition: The relevant support arrangement.

Lifecycle Condition: The dependency's lifecycle state.

Concentration Exposure: Material shared or concentrated reliance.

Transitive Risk: Relevant indirect dependency exposure.

Failure Propagation: Material disruption relationships.

External Service Exposure: Relevant service dependency conditions.

Supplier Dependency Exposure: Material supplier reliance.

Substitutability: The feasibility of effective replacement.

Replacement Feasibility: Relevant migration and transition conditions.

Lock-In Condition: Material constraints on dependency replacement.

Recovery Requirements: Conditions required for restoration.

Recovery Dependencies: Resources required during recovery.

Degradation Condition: Material deterioration affecting continued

reliance.

Failure Condition: Observed or assessed dependency failure.

Risk Identification: Material dependency risks.

Risk Assessment: Evidence-supported risk determinations.

Risk Prioritization: The significance of identified risks.

Risk Treatment Reference: Relevant treatment measures.

Residual Risk: Remaining material exposure.

Risk Acceptance Reference: Competent risk-acceptance information.

Continuity Requirements: Applicable dependency continuity expectations.

Continuity Evidence: Evidence supporting the assessment.

Evidence Competence: The suitability of evidence for the assessed claims.

Evidence Integrity: The condition of supporting evidence.

Continuity Validation: The scope and result of continuity assessment.

Continuity Uncertainty: Material unresolved conditions.

Continuity Conflicts: Contradictory continuity evidence.

Continuity Gaps: Missing material continuity information.

Critical Continuity Gaps: Material gaps requiring competent attention.

Continuity Drift: Divergence from previously recognized conditions.

Continuity Monitoring: Relevant monitoring conditions.

Continuity Reassessment: Material reassessment history.

Continuity & Risk Governance State: The recognized continuity condition.

Downstream References: Information required by competent SDS and SLGA™

governance.

The state shall not independently guarantee availability, certify security,

establish software quality, or determine legal liability.

Minimum Implementation Framework

A conforming implementation of DCRM must provide the following six functional capabilities.

• Dependency Operational Role, Criticality, and Failure Consequence

Governance

Capability to establish dependency continuity governance objects, identify

supported software functions, determine operational roles, assess dependency criticality, and recognize material failure consequences.

The implementation must distinguish component size or classification from actual operational significance.

• Availability, Maintenance, Support, and Lifecycle Governance

Capability to identify dependency availability conditions, maintenance

status, support arrangements, lifecycle state, and material assumptions concerning continued reliance.

The implementation must distinguish current functionality from supported continuity.

• Concentration, Transitive Exposure, Substitutability, and Recovery

Governance

Capability to identify shared dependency concentration, transitive risk,

failure propagation, supplier and service reliance, effective substitutability, replacement feasibility, lock-in, and recovery dependencies.

The implementation must distinguish the existence of alternatives from demonstrated replacement capability.

• Dependency Risk, Evidence, and Continuity Validation Governance

Capability to identify, assess, prioritize, and document material

dependency risks; associate continuity claims with competent evidence; preserve evidence integrity; and determine which continuity conditions are sufficiently supported.

The implementation must distinguish risk recognition from effective risk control and risk acceptance.

• Continuity Gap, Conflict, Drift, Monitoring, and Reassessment Governance

Capability to identify material continuity gaps, critical gaps, conflicting

evidence, maintenance changes, support changes, continuity drift, and justified reassessment triggers.

The implementation must preserve historical continuity states and recognize changing operational conditions.

• Governed Dependency Continuity & Risk State and SDS Handoff Governance

Capability to produce a Governed Dependency Continuity & Risk State with

sufficient criticality, availability, maintenance, concentration, substitutability, recovery, evidence, risk, uncertainty, and continuity information for competent parent SDS governance.

The implementation must not convert supported continuity into a guarantee against future dependency failure.

GIE™ Compatibility

DCRM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Dependency operational role identification

• Dependency criticality analysis

• Failure consequence mapping

• Availability condition assessment

• Maintenance status analysis

• Support condition assessment

• Dependency lifecycle monitoring

• Dependency concentration analysis

• Shared dependency exposure mapping

• Transitive risk analysis

• Failure propagation modeling

• External service dependency assessment

• Supplier dependency assessment

• Substitutability analysis

• Replacement feasibility analysis

• Dependency lock-in recognition

• Recovery dependency mapping

• Dependency degradation detection

• Dependency failure evidence association

• Risk identification and prioritization

• Risk treatment analysis

• Residual risk representation

• Continuity evidence assessment

• Continuity validation support

• Continuity uncertainty recognition

• Continuity conflict detection

• Continuity gap detection

• Critical continuity gap detection

• Continuity drift detection

• Continuity monitoring

• Continuity reassessment support

• Governed continuity state representation

GIE™ may identify candidate risks, infer possible failure propagation paths, or propose continuity assessments.

Such outputs shall remain distinguishable from sufficiently evidenced

continuity determinations.

GIE™ must not fabricate maintenance guarantees, invent supplier commitments, infer substitutability from functional similarity, or represent probabilistic risk estimates as certain future outcomes.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DCRM governance logic is:

Receive Governed Software Component Identification State

↓

Receive Governed Dependency Origin & Traceability State

↓

Receive Governed Dependency Suitability State

↓

Receive Governed Component Introduction & Change State

↓

Identify Dependency Continuity & Risk Governance Object

↓

Identify Dependent Software Object

↓

Identify Dependency Component and Relationship

↓

Identify Supported Software Functions

↓

Establish Dependency Operational Role

↓

Assess Dependency Criticality

↓

Identify Material Failure Consequences

↓

Establish Dependency Continuity Requirements

↓

Assess Dependency Availability

↓

Identify Availability Assumptions

↓

Assess Maintenance and Support Conditions

↓

Identify Dependency Lifecycle Condition

↓

Assess Dependency Concentration

↓

Identify Shared and Transitive Dependency Exposure

↓

Assess Failure Propagation

↓

Assess External Service and Supplier Dependency

↓

Assess Dependency Substitutability

↓

Assess Replacement Feasibility

↓

Identify Dependency Lock-In

↓

Identify Recovery Requirements and Recovery Dependencies

↓

Identify Degradation and Failure Conditions

↓

Identify Material Dependency Risks

↓

Assess and Prioritize Dependency Risks

↓

Identify Risk Treatment and Residual Risk

↓

Associate Continuity Evidence

↓

Assess Evidence Competence and Integrity

↓

Validate Material Continuity Conditions

↓

Identify Continuity Uncertainty

↓

Identify Continuity Conflicts

↓

Identify Continuity Gaps and Critical Gaps

↓

Identify Continuity Drift

↓

Determine Dependency Continuity & Risk Governance State

↓

Establish Governed Dependency Continuity & Risk State

↓

Preserve Continuity Monitoring and Reassessment Conditions

↓

Handoff to Parent SDS Governance

↓

Reassess When Material Dependency Conditions Change

The principal output is:

Governed Dependency Continuity & Risk State

Governance Outputs

DCRM may produce:

Dependency Continuity & Risk Governance Object

Dependent Software Function Record

Dependency Operational Role Record

Dependency Criticality Assessment

Criticality Context Record

Dependency Failure Consequence Assessment

Dependency Availability Assessment

Availability Assumption Record

Dependency Maintenance Condition Record

Dependency Support Condition Record

Dependency Lifecycle Condition Record

Dependency Maintenance Evidence Record

Dependency Concentration Assessment

Shared Dependency Exposure Map

Transitive Dependency Risk Assessment

Dependency Failure Propagation Assessment

External Service Dependency Risk Assessment

Supplier Dependency Risk Assessment

Dependency Substitutability Assessment

Replacement Feasibility Assessment

Dependency Lock-In Assessment

Dependency Recovery Requirements Record

Recovery Dependency Map

Dependency Degradation Record

Dependency Failure Record

Dependency Risk Identification Record

Dependency Risk Assessment

Dependency Risk Prioritization Record

Dependency Risk Treatment Reference

Residual Dependency Risk Record

Dependency Risk Acceptance Reference

Dependency Continuity Requirements Record

Dependency Continuity Evidence Record

Continuity Evidence Competence Assessment

Continuity Evidence Integrity Assessment

Dependency Continuity Validation Record

Dependency Continuity Uncertainty State

Dependency Continuity Conflict Record

Dependency Continuity Gap

Critical Dependency Continuity Gap

Dependency Continuity Drift Record

Dependency Continuity Change Record

Dependency Continuity Monitoring Record

Dependency Continuity Reassessment Record

Dependency Continuity & Risk Governance State

SDS Continuity & Risk Handoff Record

The principal output is:

Governed Dependency Continuity & Risk State

Critical Distinctions

DCRM preserves the following distinctions:

Current Dependency Functionality ≠ Governed Dependency Continuity

Dependency Continuity Governance Object ≠ Software Dependency Artifact

Dependency Function ≠ Dependency Criticality

Technical Component Classification ≠ Operational Dependency Role

Dependency Size ≠ Dependency Criticality

Dependency Criticality in One Context ≠ Universal Dependency Criticality

Dependency Failure ≠ Uniform Operational Consequence

Current Dependency Availability ≠ Continuity Assurance

Assumed Availability ≠ Evidenced Availability

Functional Dependency ≠ Maintained Dependency

Support Provider Existence ≠ Effective Support Assurance

Dependency Lifecycle Status ≠ Current Functional Status

Recent Release Activity ≠ Long-Term Maintenance Assurance

Multiple Dependency Relationships ≠ Independent Dependency Resilience

System Separation ≠ Dependency Independence

Indirect Dependency ≠ Indirect Operational Consequence

Dependency Connection ≠ Proven Failure Propagation

No Installed Component ≠ No External Dependency Risk

Supplier Reliance ≠ Proven Supplier Failure

Alternative Availability ≠ Effective Dependency Substitutability

Technically Feasible Replacement ≠ Authorized Dependency Replacement

Supplier Dependence ≠ Automatically Unacceptable Lock-In

Documented Recovery Plan ≠ Verified Recovery Capability

Runtime Dependency Set ≠ Recovery Dependency Set

Continued Functionality ≠ Absence of Dependency Degradation

Observed Dependency Failure ≠ Verified Failure Cause

Recognized Dependency Risk ≠ Demonstrated Risk Control

Risk Score ≠ Complete Dependency Risk Understanding

High Dependency Popularity ≠ Low Dependency Risk

Risk Treatment Proposal ≠ Authorized Risk Treatment Execution

Implemented Control ≠ Acceptable Residual Risk

Risk Assessment ≠ Risk Acceptance Authority

General Availability Expectation ≠ Defined Continuity Requirement

Continuity Evidence Presence ≠ Continuity Claim Validity

Historical Availability Evidence ≠ Future Availability Guarantee

Intact Continuity Evidence ≠ Complete Continuity Assurance

Validated Continuity Conditions ≠ Guaranteed Dependency Availability

Uncertain Continuity ≠ Proven Imminent Failure

Conflicting Continuity Evidence ≠ Permission to Select the Preferred

Conclusion

Continuity Gap ≠ Proven Dependency Failure

Critical Continuity Gap ≠ Automatic Operational Failure

Previously Acceptable Dependency Risk ≠ Permanently Acceptable Dependency

Risk

Continuity Reassessment ≠ Historical Risk Erasure

Monitoring Activity ≠ Effective Continuity Control

Continuity-Supported ≠ Dependency Failure Impossible

Cross-Architecture Boundary

DCRM governs dependency-specific continuity and risk within SDS and SLGA™.

It does not independently establish software product security, overall software quality, organization-wide business continuity, operational safety certification, or legal liability.

Within SDS:

CIM — Component Identification Module governs component identification.

DOTM — Dependency Origin & Traceability Module governs origin,

provenance, and traceability.

DSM — Dependency Suitability Module governs dependency necessity,

appropriateness, and compatibility.

CICM — Component Introduction & Change Module governs legitimate

dependency introduction and change.

DCRM — Dependency Continuity & Risk Module governs dependency

criticality, availability, maintenance, concentration, substitutability, recovery exposure, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs requirements.

Software Creation Standard (SCS) governs software creation.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change management.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

GOA™ — Governance Architecture: Authority and risk-acceptance governance.

VEGA™ — Value Execution Governance Architecture: Relevant resource

efficiency and economic sustainability governance.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

operational continuity and intervention boundaries.

DCRM preserves dependency-specific continuity and risk governance without

replacing the competent responsibilities of these domains.

Canonical Boundary

DCRM begins when a material software dependency relationship requires assessment of its operational criticality, availability, maintenance, support, substitutability, disruption exposure, recovery conditions, or continued reliance.

DCRM governs:

- Dependency continuity governance objects
- Dependent software functions
- Dependency operational roles
- Dependency criticality
- Criticality context
- Failure consequences
- Dependency availability
- Availability assumptions
- Maintenance condition
- Support condition
- Dependency lifecycle condition
- Maintenance evidence
- Dependency concentration
- Shared dependency exposure
- Transitive dependency risk
- Failure propagation
- External service dependency risk
- Supplier dependency risk
- Dependency substitutability
- Replacement feasibility
- Dependency lock-in
- Recovery requirements
- Recovery dependencies
- Dependency degradation
- Dependency failure
- Dependency risk identification
- Dependency risk assessment
- Risk prioritization

• Risk treatment references

• Residual risk

• Risk acceptance references

• Continuity requirements

• Continuity evidence

• Evidence competence

• Evidence integrity

• Continuity validation

• Continuity uncertainty

• Continuity conflicts

• Continuity gaps

• Critical continuity gaps

• Continuity drift

• Continuity changes

• Continuity monitoring

• Continuity reassessment

• Continuity and risk governance states

DCRM does not independently determine:

• Complete component identification

• Complete dependency provenance

• Dependency suitability

• Dependency introduction authorization

• Software security certification

• Software product quality

• Organization-wide business continuity

• Software release readiness

• Deployment authorization

• Legal liability

DCRM ends when the relevant dependency continuity and risk conditions have been represented as a:

Governed Dependency Continuity & Risk State

with sufficient criticality, availability, maintenance, support,

concentration, substitutability, recovery, evidence, risk, uncertainty, and continuity information for competent parent SDS and downstream SLGA™ governance.

Use Case 1 — A Critical Software System Depends on an Abandoned Library

That Still Works

An organization operates a software platform supporting essential

operational processes.

The platform has been functioning reliably for several years.

One relatively small external library performs a critical communication

function.

The library was introduced during the original development process.

Its current version remains technically functional.

The organization has not experienced a major failure involving the

component.

However, the library's original maintainers have discontinued support.

No new releases are planned.

The organization has not established a sufficiently supported replacement

strategy.

The library remains embedded in the software platform.

The organization continues operating because the dependency still works.

Over time, the platform's runtime environment changes.

The library becomes incompatible with a required infrastructure update.

The critical communication function fails.

The organization discovers that the dependency cannot be replaced quickly without material software modifications.

DCRM evaluates:

Dependency Identity → Operational Role → Criticality → Maintenance

Condition → Substitutability → Recovery Requirements → Continuity Risk

The assessment identifies:

• The dependency supported a critical software function

• The component remained functional despite discontinued maintenance

• Its continued operation depended on increasingly fragile compatibility

• Effective replacement feasibility had not been established

• The organization lacked sufficient evidence of sustainable dependency

conditions. continuity.

Before the failure, the relevant governance state could have been:

Continuity-at-Risk

or:

Substitutability-Unresolved

After the disruption, the state may become:

Disrupted

The critical distinctions are:

Functional Dependency ≠ Maintained Dependency.

Dependency Size ≠ Dependency Criticality.

Alternative Availability ≠ Effective Dependency Substitutability.

Current Dependency Functionality ≠ Governed Dependency Continuity.

The result: DCRM reveals that a dependency can remain technically functional while its operational continuity deteriorates, and that continuity governance must recognize critical reliance before a dependency failure exposes its consequences.

Use Case 2 — Multiple AI Services Depend on One External Provider

Without Recognized Concentration Risk

An organization operates several AI-enabled software services.

The services support:

• Customer communication

• Internal document processing

• Automated workflow execution

• Software development assistance

• Operational decision support

Each service has its own application architecture and development team.

The services appear operationally independent.

However, several critical functions rely on the same external software

service provider.

The provider supplies authentication, API access, and a shared execution

platform used across multiple applications.

The organization's software dependency records identify the provider within individual systems.

They do not establish the combined concentration of operational reliance.

The provider experiences a prolonged service disruption.

Multiple organizational functions become unavailable at the same time.

The organization discovers that alternative providers exist but cannot be integrated without substantial software changes.

The services were separated at the application level.

Their critical dependency exposure was concentrated.

DCRM evaluates:

Individual Dependency Relationships → Shared Provider → Concentration →

Failure Propagation → Substitutability → Recovery Requirements → Continuity State

The assessment identifies:

• Multiple software systems depended on the same provider

• The provider represented a shared operational dependency

• Application separation did not establish dependency independence

• Alternative providers were not immediately substitutable

• Recovery required changes across multiple software systems

• The combined dependency concentration had not been sufficiently

governed.

Before the disruption, the relevant governance state could have been:

Continuity-at-Risk

or:

Recovery-Unresolved

Following the disruption, affected dependency relationships may be recognized as:

Disrupted

The critical distinctions are:

System Separation ≠ Dependency Independence.

Multiple Dependency Relationships ≠ Independent Dependency Resilience.

Alternative Availability ≠ Effective Dependency Substitutability.

Documented Recovery Plan ≠ Verified Recovery Capability.

The result: DCRM makes shared dependency concentration visible before a

single external failure becomes a simultaneous disruption across otherwise separate software systems.

Canonical Closing Statement

Modern software does not operate independently of the resources upon which it relies.

Applications depend on libraries.

Libraries depend on other libraries.

Services depend on platforms.

Platforms depend on infrastructure.

Build systems depend on repositories, tools, and distribution mechanisms.

Autonomous systems may depend on external services whose continued availability is outside their direct control.

A software system may be technically correct, securely configured, and fully functional today.

Yet its operation tomorrow may depend on a component whose maintenance has ended, a provider that cannot be replaced, a service whose availability is uncertain, or a transitive dependency whose criticality has never been recognized.

Current Dependency Functionality ≠ Governed Dependency Continuity.

Dependency Size ≠ Dependency Criticality.

Functional Dependency ≠ Maintained Dependency.

Alternative Availability ≠ Effective Dependency Substitutability.

System Separation ≠ Dependency Independence.

Historical Availability Evidence ≠ Future Availability Guarantee.

DCRM establishes a fundamental governance requirement:

Software dependency reliance must remain justified not only by what a dependency does today, but by the conditions under which that dependency can continue supporting the software system—and by the consequences when those conditions cease to hold.

This requirement applies regardless of whether the dependency is internal, external, direct, transitive, generated, embedded, or provided through a remote service.

It applies regardless of whether the dependent software was created by

human developers, automated tools, AI coding assistants, or autonomous development agents.

The technology may change.

The dependency network may expand.

The provider may change.

The software may become more autonomous.

The requirement to govern operational reliance remains.

DCRM does not promise that dependencies will never fail.

It does not confuse risk assessment with risk elimination.

It does not assume that a recovery plan guarantees recovery.

And it does not treat present functionality as proof of sustainable operation.

Instead, it establishes the conditions under which material dependency

risks, criticality, concentration, maintenance, substitutability, and recovery limitations can be recognized, evidenced, and reassessed before or after disruption.

DCRM transforms software dependency continuity from an assumption of

continued functionality into a governed relationship of operational reliance — ensuring that software systems can identify not only what they depend upon, but what happens when that dependency weakens, changes, becomes unavailable, or can no longer be sustained.

A dependency is not continuity-governed because it still works. It is continuity-governed when the conditions of continued reliance, the consequences of disruption, the limits of replacement, and the evidence supporting those determinations remain visible, assessable, and subject to competent governance throughout the software lifecycle.